

simple example to get data from web and received data as json and then convert that data into python dict

Here's a **simple, clean example** to:

1. Fetch data from a web API
2. Receive it as JSON
3. Convert it into a Python dictionary

✅ **Example using requests**

◆ **Step 1: Install requests (if not installed)**

pip install requests

◆ **Step 2: Python Code**

import requests

API URL (dummy free API)

url = "https://jsonplaceholder.typicode.com/users/1"

Send GET request

response = requests.get(url)

Convert JSON response to Python dictionary

data = response.json()

Print the dictionary

print(data)

Access specific values

print("Name:", data["name"])

print("Email:", data["email"])

◆ **Output (Example)**

```
{
  'id': 1,
  'name': 'Leanne Graham',
  'username': 'Bret',
  'email': 'Sincere@april.biz',
```

```
...  
}
```

Key Concept

- `response.json()`  converts JSON → Python dict
- JSON = text format
- Python dict = usable object

Simple Breakdown

Step	What Happens
API Call	<code>requests.get(url)</code>
Response	JSON (string format)
Conversion	<code>.json()</code> → dict
Usage	<code>data["name"]</code>

Bonus (Check type)

```
print(type(data)) # <class 'dict'>
```

If you want, I can next show:

- POST API example
- Error handling (important for real projects)
- How to use this in your ERP system 

base upon above just generate 10 easy to medium questions with answers

Here are **10 easy → medium practice questions with answers** based on fetching JSON data from a web API and converting it into a Python dictionary.

1. Fetch and print full JSON data

Question:

Write a program to fetch data from an API and print the full response as a dictionary.

Answer:

```
import requests

response = requests.get("https://jsonplaceholder.typicode.com/posts/1")
data = response.json()

print(data)
```

✅ 2. Print specific field

Question:

Fetch user data and print only the name.

Answer:

```
import requests

data = requests.get("https://jsonplaceholder.typicode.com/users/1").json()
print(data["name"])
```

✅ 3. Check response status

Question:

Check if API call is successful before converting to JSON.

Answer:

```
import requests

response = requests.get("https://jsonplaceholder.typicode.com/users/1")

if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print("Request failed")
```

✅ 4. Print multiple fields

Question:

Print name, email, and phone from API response.

Answer:

```
import requests

data = requests.get("https://jsonplaceholder.typicode.com/users/1").json()

print(data["name"])
```

```
print(data["email"])
print(data["phone"])
```

✓ 5. Loop through list data

Question:

Fetch all users and print their names.

Answer:

```
import requests
```

```
data = requests.get("https://jsonplaceholder.typicode.com/users").json()
```

```
for user in data:
    print(user["name"])
```

✓ 6. Count total records

Question:

Fetch all posts and print total number of posts.

Answer:

```
import requests
```

```
data = requests.get("https://jsonplaceholder.typicode.com/posts").json()
```

```
print(len(data))
```

✓ 7. Filter data

Question:

Print users whose id is greater than 5.

Answer:

```
import requests
```

```
data = requests.get("https://jsonplaceholder.typicode.com/users").json()
```

```
for user in data:
    if user["id"] > 5:
        print(user["name"])
```

✓ 8. Handle missing key safely

Question:

Print company name safely (avoid error if key not present).

Answer:

```
import requests
```

```
data = requests.get("https://jsonplaceholder.typicode.com/users/1").json()
```

```
company = data.get("company", {}).get("name", "Not Available")  
print(company)
```

✅ 9. Convert JSON string manually

Question:

Convert JSON string into dictionary using Python.

Answer:

```
import json
```

```
json_data = '{"name": "Kamal", "age": 25}'
```

```
data = json.loads(json_data)
```

```
print(data)
```

✅ 10. Pretty print JSON

Question:

Print API response in readable format.

Answer:

```
import requests
```

```
import json
```

```
data = requests.get("https://jsonplaceholder.typicode.com/users/1").json()
```

```
print(json.dumps(data, indent=4))
```

🔥 If you want next level:

I can generate:

- 🔴 Hard questions (with nested JSON)
- 🔴 Real-world API tasks (like payment / GST / ERP)
- 🔴 Mini project based on API handling

Just tell 👍

30 practice questions easy to advanced

Here are **30 practice questions (Easy → Advanced)** based on **fetching web data (JSON API)** and **converting to Python dict + processing it**.

 EASY LEVEL (1–10)

- 1.**
Fetch data from an API and print full JSON response.
 - 2.**
Convert API response into a Python dictionary.
 - 3.**
Print only the name from user data API.
 - 4.**
Print email and username from API response.
 - 5.**
Check response status code before processing JSON.
 - 6.**
Count total users returned by API.
 - 7.**
Print all user names from API.
 - 8.**
Fetch a single post and print its title.
 - 9.**
Convert JSON string into Python dictionary.
 - 10.**
Pretty print JSON response using indentation.
-

 MEDIUM LEVEL (11–20)

- 11.**
Print users whose id is greater than 5.
- 12.**
Find user whose name contains "Leanne".
- 13.**
Extract all emails from user list API.
- 14.**
Print only users who have company details.
- 15.**
Handle missing keys using `.get()` safely.
- 16.**
Fetch posts and print only first 5 titles.
- 17.**

Count how many posts exist in API.

18.

Filter posts where userId = 1.

19.

Create a list of all usernames from API.

20.

Check if API response contains empty data or not.

● **INTERMEDIATE LEVEL (21–25)**

21.

Fetch users and group them by company name.

22.

Create a dictionary where key = user id and value = name.

23.

Fetch posts and store them in a local Python list.

24.

Find the user with maximum ID.

25.

Combine data from two APIs (users + posts).

● **ADVANCED LEVEL (26–30)**

26.

Build a function that fetches API data and returns Python dict safely (with error handling).

27.

Retry API request if it fails (basic retry mechanism).

28.

Fetch users and create a report:

👉 name, email, company name in structured format.

29.

Build a mini "search engine" that searches users by name from API data.

30.

Design a caching system:

👉 If data already fetched once, don't call API again, use stored dictionary.

🔥 **Bonus Challenge Idea (for your students)**

Build a mini project:

👉 "User Management System using API data"

Features:

- Fetch users
- Search user

- Filter users
- Save locally
- Display formatted output

If you want next level, I can also generate:

- ✓ Answer sheet for all 30
- ✓ Mini project solution (complete code)
- ✓ Real-world ERP API simulation (like GST system)